AI Practical Programs Reference

Prolog & Python Code Implementations

1. Prolog: Family Tree

PROLOG

```
parent(john, mary).
parent(john, tom).
male(john).
male(tom).
female(mary).

father(X, Y) :- parent(X, Y), male(X).
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

2. Prolog: Even/Odd Check

PROLOG

```
even(X) :- 0 is X mod 2.
odd(X) :- 1 is X mod 2.
```

3. Python: BFS – Uninformed Search

PYTHON

```
from collections import deque

def bfs(graph, start):
    visited, queue = set(), deque([start])
    while queue:
        vertex = queue.popleft()
        if vertex not in visited:
            visited.add(vertex)
            print(vertex, end=" ")
            queue.extend(graph[vertex] - visited)

# Sample Graph
graph = {'A': {'B', 'C'}, 'B': {'A', 'D', 'E'}, 'C': {'A', 'F'}, 'D': {'B'},
'E': {'B'}, 'F': {'C'}}
bfs(graph, 'A')
```

4. Prolog: Divisibility by given numbers like 2, 4, 5, 7, 8

PROLOG

```
divisible(N, D) :- 0 is N mod D.
check_div(N) :- divisible(N, 2), divisible(N, 4), divisible(N, 5),
divisible(N, 7), divisible(N, 8).
```

5. Python: DFS – Uninformed Search

PYTHON

```
def dfs(graph, node, visited=None):
    if visited is None: visited = set()
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for n in graph[node] - visited:
            dfs(graph, n, visited)

graph = {'A': {'B', 'C'}, 'B': {'A', 'D', 'E'}, 'C': {'A', 'F'}, 'D': {'B'},
'E': {'B'}, 'F': {'C'}}
dfs(graph, 'A')
```

6. Prolog: LCM and GCD

PROLOG

```
gcd(X, 0, X) :- !.
gcd(X, Y, Z) :- R is X mod Y, gcd(Y, R, Z).
lcm(X, Y, Z) :- gcd(X, Y, G), Z is abs(X * Y) // G.
```

7. Python: McCulloch-Pitts – AND, OR, XOR

PYTHON

```
def mp_neuron(inputs, weights, threshold):
    return 1 if sum(i*w for i, w in zip(inputs, weights)) >= threshold else
0

inputs = [(0,0), (0,1), (1,0), (1,1)]

print("AND:", [mp_neuron(x, [1, 1], 2) for x in inputs])
print("OR: ", [mp_neuron(x, [1, 1], 1) for x in inputs])

# XOR requires a 2-layer approach (NOT (x1 AND x2) AND (x1 OR x2))
def xor_mp(x1, x2):
    z1 = mp_neuron((x1, x2), (1, -1), 1) # x1 AND NOT x2
    z2 = mp_neuron((x1, x2), (-1, 1), 1) # NOT x1 AND x2
    return mp_neuron((z1, z2), (1, 1), 1) # z1 OR z2

print("XOR:", [xor_mp(*x) for x in inputs])
```

8. Prolog: Calculator

PROLOG

```
add(X, Y, Z) :- Z is X + Y.
sub(X, Y, Z) :- Z is X - Y.
mul(X, Y, Z) :- Z is X * Y.
div(X, Y, Z) :- Y \= 0, Z is X / Y.
```

9. Python: Perceptron Learning Algorithm

PYTHON

```
import numpy as np

X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1]) # Target: AND Logic

w, b, lr = np.array([0.0, 0.0]), 0.0, 0.1

for _ in range(10): # Epochs
    for xi, yi in zip(X, y):
        y_pred = 1 if np.dot(xi, w) + b >= 0 else 0
        update = lr * (yi - y_pred)
        w += update * xi
        b += update

print("Weights:", w, "Bias:", b)
```

10. Python: Delta Learning Algorithm

PYTHON

```
import numpy as np

X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1])

w = np.array([0.1, 0.1])
lr = 0.1

for _ in range(20): # Epochs
    for xi, yi in zip(X, y):
        y_pred = np.dot(xi, w) # Linear activation for Delta rule
        error = yi - y_pred
        w += lr * error * xi

print("Delta Weights:", w)
```

11. Python: A* Search Algorithm

PYTHON

```
import heapq

def a_star(graph, start, goal, h):
    open_set, g_score = [(0, start)], {start: 0}

    while open_set:
        _, curr = heapq.heappop(open_set)
        if curr == goal: return g_score[curr]

        for neighbor, cost in graph.get(curr, []):
            tentative_g = g_score[curr] + cost
            if tentative_g < g_score.get(neighbor, float('inf')):
                g_score[neighbor] = tentative_g
                heapq.heappush(open_set, (tentative_g + h[neighbor],
neighbor))
    return None

# Graph: Node -> [(Neighbor, Path_Cost)]
graph = {'A': [('B',1), ('C',3)], 'B': [('D',1)], 'C': [('D',1)]}
heuristics = {'A': 2, 'B': 1, 'C': 1, 'D': 0}

print("Shortest path cost to D:", a_star(graph, 'A', 'D', heuristics))
```

12. Prolog: Prime Number Check

PROLOG

```
has_factor(N, L) :- N mod L =:= 0.
has_factor(N, L) :- L * L < N, L2 is L + 2, has_factor(N, L2).

is_prime(2).
is_prime(3).
is_prime(N) :- integer(N), N > 3, N mod 2 =\= 0, \+ has_factor(N, 3).
```

13. Prolog: Factorial

PROLOG

```
fact(0, 1).
fact(N, F) :- N > 0, N1 is N - 1, fact(N1, F1), F is N * F1.
```